C++ Code: Arrays, Searching and Sorting

Data Structures Lab | 0-based indexing | `n` is the number of elements

1. Array Operations

```
#include <iostream>
using namespace std;

void traverse(const int A[], int n) {
    for (int i = 0; i < n; i++)
        cout << A[i] << " ";
    cout << endl;
}

int insertAt(int A[], int n, int pos, int item) {   // returns new size
    for (int j = n - 1; j >= pos; j--)
        A[j + 1] = A[j];                            // shift right
    A[pos] = item;
    return n + 1;
}

int deleteAt(int A[], int n, int pos) {             // returns new size
    for (int j = pos; j < n - 1; j++)
        A[j] = A[j + 1];                            // shift left
    return n - 1;
}
```

2. Linear Search

```
int linearSearch(const int A[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (A[i] == key)
            return i;
    return -1;                                      // not found
}
```

3. Binary Search

Iterative

```
int binarySearch(const int A[], int n, int key) {   // A must be sorted
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (A[mid] == key)      return mid;
        else if (A[mid] < key)  low = mid + 1;
        else                    high = mid - 1;
    }
    return -1;
}
```

Recursive

```
int binarySearchRec(const int A[], int low, int high, int key) {
    if (low > high)
        return -1;
    int mid = low + (high - low) / 2;
    if (A[mid] == key)      return mid;
    else if (A[mid] < key)  return binarySearchRec(A, mid + 1, high, key);
    else                    return binarySearchRec(A, low, mid - 1, key);
}
// first call: binarySearchRec(A, 0, n - 1, key);
```

4. Bubble Sort

```
#include <algorithm>   // std::swap

void bubbleSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (A[j] > A[j + 1]) {
                swap(A[j], A[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;                        // already sorted
    }
}
```

5. Insertion Sort

```
void insertionSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        int key = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > key) {
            A[j + 1] = A[j];                        // shift right
            j--;
        }
        A[j + 1] = key;
    }
}
```

6. Selection Sort

```
void selectionSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (A[j] < A[minIdx])
                minIdx = j;
        swap(A[i], A[minIdx]);
    }
}
```